

Project Part A

1 Search Strategy: A* Algorithm

1.1 Reason For Choosing

The **A* search** algorithm was chosen because, as stated in the course theorem, A* search is optimal and complete when using an admissible heuristic. The state space expands rapidly due to the three action choices (Move, Eat, Cascade). Uninformed search algorithms, such as **Breadth-First Search** or **Depth-First Search**, would quickly exceed memory or time limits.

Test Case	Opt. depth	BFS nodes	BFS time	A* nodes	A* time	Node reduction
vis1	2	2	0.12ms	7	0.34ms	0.3×
vis2	2	3	0.21ms	5	0.42ms	0.6×
custom-1	5	212	7.20ms	37	1.82ms	5.7×
custom-2	13	37,227	9,059ms	17,379	5.656ms	2.1×
custom-3	24	101,594	6,108ms	32,278	2,532ms	3.1×

*vis1, 2 : given test scenario

*custom-1 : R1 at (3,0), R1 at (3,1), and B2 at (3,5)

*custom-2 : R3 at (6,0) with B1 at (0,7) and B2 at (1,7)

*custom-3 : R2 at (3,3) with B1 stacks at all four corners and at (0,3)

The above table compares A* with the admissible heuristic against BFS across five test cases spanning solution depths 2 to 24. (All measurements are averaged over 5 runs on the same hardware. A* returns the same solution length as BFS in every case, providing empirical confirmation that the heuristic does not compromise solution optimality on any of the tested inputs.

For shallow solutions (depth 2), A* explores slightly more nodes than BFS due to the overhead of maintaining the priority queue and computing $h(n)$ at each node. A* consistently maintains a smaller search space than BFS as the problem complexity increases. The degree of node reduction varies by test case, but the heuristic effectively prevents the exponential node explosion inherent in uninformed search.

1.2 Heuristic Function

The heuristic $h(n)$ estimates the cost to eliminate all remaining **Blue** cells. For a set of **Red** cells R and **Blue** cells B , the cost is defined as:

$$h(n) = \max_{b \in B} \left(\min_{r \in R} \left\lceil \frac{D(r, b)}{H_r} \right\rceil \right)$$

where $D(r, b)$ is the Manhattan distance between cells r and b , and H_r is the height of the **Red** stack. This represents the minimum actions required for the most ‘distant’ target under ideal conditions.

The heuristic function derives from a trivial one, which is the Manhattan distance between cells r and b $D(r, b)$. However, because the action ‘Cascade’ will reduce the cost when playing, we want the heuristic to be closer to the actual cost. So we introduce the height of the **Red** stacks to the formula.

Other than that, for a **Blue** stack, we want to find its ‘nearest’ **Red** stack, so we selected $\min_{r \in R}$. Overall, we want all the **Blue** stacks to be eliminated, so the progress of the game depends on the **Blue** stack with the biggest eliminating cost. In order not to overestimate ($h \leq h^*$), we selected $\max_{b \in B}$

1.3 Admissible Proof

As explained in the lecture, a heuristic is admissible if $h(n) \leq h^*(n)$, meaning it never overestimates the true cost to the goal. We define the true minimum cost as $C^*(r, b)$. Any Move or Eat action costs 1 step, but the cost of Cascade depends on the height of the stack and the Manhattan distance. In a border case, $D(r, b) = H_r$, where only with 1 step can the **Red** eliminate **Blue**. Therefore, for a specific **Red** stack to eliminate a specific **Blue** cell is bounded by:

$$C^*(r, b) \geq \left\lceil \frac{D(r, b)}{H_r} \right\rceil$$

For a specific **Blue** cell b , the most efficient way to eliminate it is to use the **Red** stack $r \in R$ that requires the fewest actions. Thus, the true cost to eliminate b $h^*(n, b)$, must satisfy:

$$h^*(n, b) = \min_{r \in R} c^*(r, b) \geq \min_{r \in R} \left\lceil \frac{D(r, b)}{H_r} \right\rceil$$

The goal is reached only when all **Blue** cells B are eliminated. The total cost to clear the board $h^*(n)$ cannot be less than the cost required to eliminate the single most difficult-to-reach **Blue** cell.

$$h^*(n) \geq \max_{b \in B} \left(\min_{r \in R} \left\lceil \frac{D(r, b)}{H_r} \right\rceil \right) = h(n)$$

1.4 Implementation Detail

We used some data structures and state transformation to facilitate the A* search process:

- **State Representation:** The **dict** for coordinate lookups is converted into **frozenset** objects for state tracking. Each state is represented as a **frozenset** of (**Coord**, **Color**, **Height**) tuples, providing an immutable and hashable signature.
- **Priority Queue Management:** The **open_set** is implemented using Python's **heapq** module, maintaining a min-heap based on the total cost $f(n) = g(n) + h(n)$. To handle ties in f -scores, a unique **counter** is inserted into the heap tuples: (**f**, **counter**, **g**, **board**, **actions**). This prevents illegal comparisons between **dict** objects and ensures deterministic tie-breaking.
- **Cycle Pruning:** A **visited** set (Closed List) stores the **frozenset** representation of every expanded board. Any newly generated state that already exists in this set is immediately discarded.

1.5 Time & Space Complexity Analysis

Let b be the branching factor ($\leq 12|R|$), d the optimal solution depth, $|R|$ the number of **Red** stacks, and $|B|$ the number of **Blue** target cells. $N = |R| + |B|$ is the total number of pieces on the board.

- **Time Complexity:** $O(b^d \cdot |R||B|)$
The search space grows exponentially as $O(b^d)$. At each node, computing the heuristic over all **Red-Blue** pairs takes $O(12|R||B|)$ time.
- **Space Complexity:** $O(b^d \cdot N)$
A* stores all generated states to ensure optimality and avoid cycles. Each state requires $O(N)$ space to hold the positions and properties of all pieces, making memory the main limitation in deep or complex searches.

2 Moving to a Simultaneous Action Choice

2.1 Changes Caused

1. The cost metric changes.

In the original model, the path cost was the number of individual actions performed. Under the simultaneous model, the path cost becomes the number of turns, where each turn comprises one action per **Red** stack. The optimal solution is now defined as the sequence of turns that eliminates all **Blue** tokens in the minimum number of turns. Since all **Red** stacks act within a single turn, fewer turns may suffice compared to the number of actions required sequentially.

2. The branching factor grows exponentially.

Originally, with n **Red** stacks each having b possible actions, the branching factor was $n \times b$. Under simultaneous execution the branching factor is now up to b^n . Joint actions in which two stacks target the same cell or attempt to eat the same **Blue** will have to be filtered out.

2.2 Potential Solution

To accommodate this, the primary code change is replacing **get_possible_actions()** with a joint-action generator that enumerates all valid combinations of per-stack actions. The state representation, goal test, and the A* search loop require no modification. This is because the state is still a board configuration and the search structure is unchanged.